

SmallScaleAES 差分密码分析

项目运行手册

Differential Analysis Project

目录

1 项目概述

本项目实现了 SmallScaleAES (nibble 级简化 AES) 上的差分密码分析, 包含两个核心模块:

- **差分区分器**: 验证 3 轮差分 trail ($\Delta P \rightarrow \Delta C^*$, 概率 2^{-18}), 通过 $N = 2^{20}$ 个明文对统计命中数, 与随机预言机对比
- **密钥恢复**: 利用 Modified 2.5+1 轮差分 trail (前 8 个 S-box 替换为恒等, 概率 2^{-26}), 通过 $N = 2^{26}$ 个明文对恢复最后一轮 4 个活跃位置的子密钥

2 系统要求

- **编译器**: g++ 9.0+ (需支持 C++17)
- **操作系统**: Linux (Ubuntu 22.04+推荐)
- **依赖**: 无外部库依赖 (仅使用 C++ 标准库)

3 项目目录结构

```
differential_code/
|-- project/                                # 文档交付
|   |-- theory.pdf / .tex                   # 理论基础
|   +-- manual.pdf / .tex                   # 本运行手册
|-- tex/                                    # TikZ 图表源文件
|   |-- differential_3round_trail.tex        # 3轮差分 trail 图
|   |-- differential_4r_keyrecovery.tex     # 4轮密钥恢复图
|   |-- differential_cryptanalysis.tex      # 差分分析总览图
|   |-- differential_hypothesis_test.tex    # 假设检验图
|   +-- differential_25round_trail.tex      # 多轮 trail 扩展图
|-- SmallScaleAES.h / .cpp                  # 实现模块: SmallScaleAES 密码核心
|-- distinguisher.h                         # 接口: 差分区分器
|-- distinguisher_lib.cpp                   # 实现模块: 区分器核心逻辑
|-- key_recovery.h                         # 接口: 密钥恢复
|-- key_recovery_lib.cpp                   # 实现模块: 密钥恢复核心逻辑
|-- DifferentialDistinguisher.cpp          # 独立演示: 区分器完整程序
|-- KeyRecovery_Modified_Simple.cpp       # 独立演示: 密钥恢复完整程序
|-- test_correctness.cpp                  # 测试模块: 正确性测试
|-- bench_perf.cpp                       # 测试模块: 性能基准
+-- Makefile
```

4 构建与运行

4.1 正确性测试

```
cd differential_code
make test
```

测试内容：

1. SmallScaleAES 加密/解密往返验证
2. encrypt_noMC_rounds 与 encrypt 等价性 (5 轮)
3. 差分区分器产生命中 (验证 trail 非平凡)
4. 命中数在泊松期望范围内 ($\lambda = 4$)
5. 随机预言机产生零命中 (验证非零输出差分的稀疏性)
6. Modified 加密的确定性 (相同输入产生相同输出)
7. 子密钥位置与密钥扩展的一致性

4.2 性能基准

```
cd differential_code
make bench
```

基准内容：

- 区分器：3 次试验 (每次 2^{20} 对) 的计时
- 密钥恢复： $N = 2^{16}$ 对、全 65536 子密钥搜索的计时

4.3 独立演示程序

```
# 运行完整差分区分器演示 (20 次试验, Poisson 直方图)
g++ -O3 -std=c++17 -o dist_demo \
    SmallScaleAES.cpp DifferentialDistinguisher.cpp
./dist_demo 20

# 运行密钥恢复演示 (N=2~26 对)
g++ -O3 -std=c++17 -o kr_demo \
    SmallScaleAES.cpp KeyRecovery_Modified_Simple.cpp
./kr_demo 26 1
```

参数说明：

- argv[1]: N 的对数 ($N = 2^{arg}$), 默认 26 (即 $N = 2^{26}$)
- argv[2]: 阈值 η , 默认 1

5 典型输出示例

5.1 正确性测试输出

```
$ make test
=== Differential Analysis Correctness Tests ===

[PASS] SmallScaleAES encrypt/decrypt roundtrip
[PASS] encrypt_noMC_rounds == encrypt (5 rounds)
[PASS] Distinguisher produces hits (trial 1)
[PASS] Distinguisher hits near lambda
[PASS] Random oracle produces 0 hits
[PASS] Modified encryption deterministic
[PASS] Subkey positions consistent with key schedule

=== Results: 7 passed, 0 failed ===
```

5.2 密钥恢复演示输出

```
$ ./kr_demo 26 1
Modified 2.5+1 Round Differential Key Recovery
Parameters:
  N (pairs)   = 2^26 = 67108864
  Threshold   = 1
  Expected lambda = N * p = 1

Key Recovery Results
Candidates with count >= 1: 7

Correct subkey: (K_4[0], K_4[13], K_4[10], K_4[7]) = (5, 9, e, 1)
Correct subkey count: 3

Correct subkey found at rank 1!

Top 20 candidates:
Rank | Subkey (K_4[0,13,10,7]) | Count | Correct?
-----+-----+-----+-----
  1 | (5, 9, e, 1)             |      3 | YES
  2 | (1, 7, 5, e)             |      2 |
  ...
```